Fundamentals of Object Oriented Programming: Object oriented paradigm, Object and Classes, Data Abstraction and Encapsulation, Inheritance, Polymorphism, Dynamic Binding.

1. Sample Case study on class & objects CO1

Bank Account Management System:

Problem Statement: Design a system to manage bank accounts.

Classes: Account, Savings-Account, Current-Account.

Attributes: Account number, balance, account type, interest rate (for savings account), overdraft limit (for current account), etc.

Methods: Deposit, Withdraw, Calculate Interest, Transfer, etc.

Object Usage: Create objects for each customer's account and perform transactions like deposits, withdrawals, and transfers.

Algorithm: Bank Management System

Step 1: Start the program.

Step 2: Create a `BankAccount` class with account number, account holder name, and balance.

Step 3: Create a constructor to initialize the account details.

Step 4: Create a `deposit()` method to add money to the account.

Step 5: Create a `withdraw()` method to withdraw money after checking the balance.

Step 6: Create a `displayDetails()` method to display account information.

Step 7: Create getter methods to access the account number and balance.

Step 8: Create a `Bank` class containing an array of `BankAccount` objects.

Step 9: Create a `createAccount()` method to create a new bank account.

Step 10: Create a `findAccount()` method to search for an account using the account number.

Step 11: Create a `depositMoney()` method to deposit money into an existing account.

Step 12: Create a `withdrawMoney()` method to withdraw money from an existing account.

Step 13: Create a `checkBalance()` method to display the current balance.

Step 14: Create a `displayAccount()` method to display complete account details.

Step 15: In the `main()` method, create a `Scanner` object and a `Bank` object.

Step 16: Display the Bank Management System menu:

1. Create Account
2. Deposit Money
3. Withdraw Money
4. Check Balance
5. Display Account Details
6. Exit

Step 17: Read the user's choice.

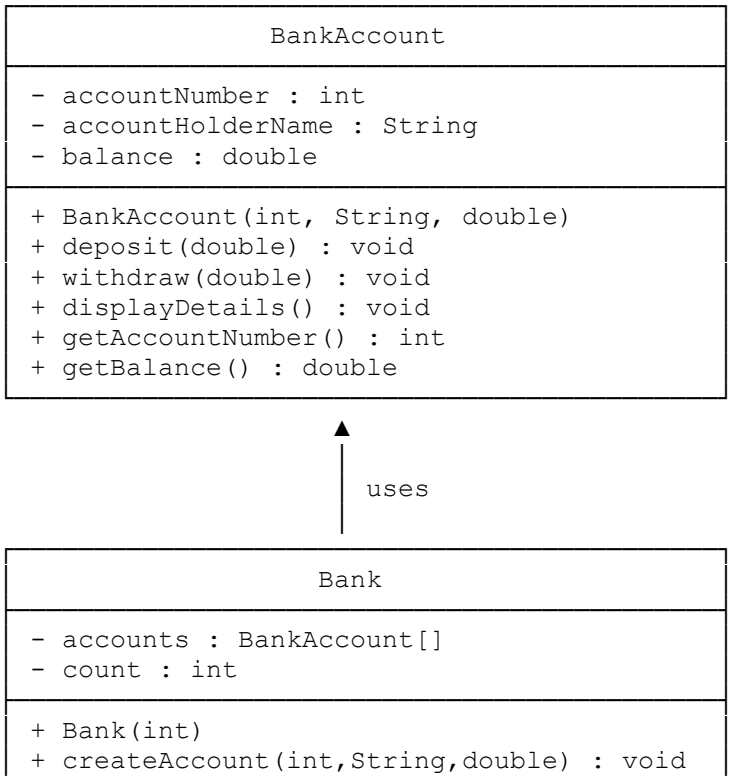
Step 18: Perform the selected operation using a `switch` statement.

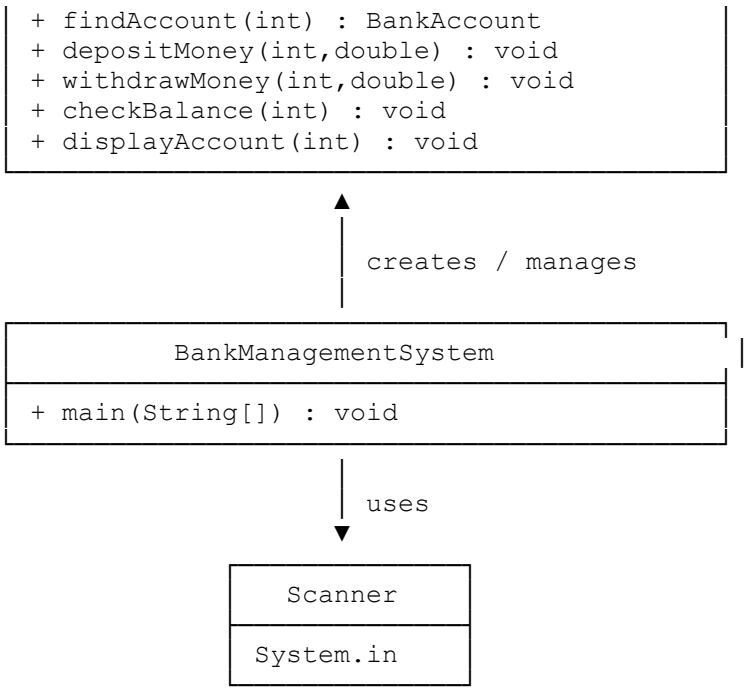
Step 19: Repeat the menu until the user selects option 6.

Step 20: Close the `Scanner`.

Step 21: Stop the program.

UML Class Diagram





Code:

```java
package javacore;
import java.util.Scanner;

// Bank Account Class
class BankAccount {

    private int accountNumber;
    private String accountHolderName;
    private double balance;

    // Constructor
    BankAccount(int accountNumber, String accountHolderName,
double balance) {
        this.accountNumber = accountNumber;
        this.accountHolderName = accountHolderName;
        this.balance = balance;
    }

    // Deposit method
    public void deposit(double amount) {
```

```
        if (amount > 0) {
            balance = balance + amount;
            System.out.println("Amount deposited
successfully.");
        } else {
            System.out.println("Invalid amount.");
        }
    }

    // Withdraw method
    public void withdraw(double amount) {
        if (amount <= 0) {
            System.out.println("Invalid amount.");
        } else if (amount > balance) {
            System.out.println("Insufficient balance.");
        } else {
            balance = balance - amount;
            System.out.println("Amount withdrawn
successfully.");
        }
    }

    // Display account details
    public void displayDetails() {
        System.out.println("\n----- Account Details -----");
        System.out.println("Account Number : " +
accountNumber);
        System.out.println("Account Holder : " +
accountHolderName);
        System.out.println("Balance        : Rs. " + balance);
    }

    // Getter for account number
    public int getAccountNumber() {
        return accountNumber;
    }
```

```
    // Getter for balance
    public double getBalance() {
        return balance;
    }
}


// Bank Management Class
class Bank {

    // Fixed-size array instead of ArrayList
    private BankAccount[] accounts;
    private int count;

    // Constructor
    Bank(int size) {
        accounts = new BankAccount[size];
        count = 0;
    }

    // Create account
    public void createAccount(int accountNumber, String name,
double initialBalance) {

        if (count >= accounts.length) {
            System.out.println("Bank account limit reached.");
            return;
        }

        if (findAccount(accountNumber) != null) {
            System.out.println("Account already exists.");
            return;
        }

        if (initialBalance < 0) {
```

```
            System.out.println("Invalid initial balance.");
            return;
        }

        accounts[count] =
            new BankAccount(accountNumber, name,
initialBalance);

        count++;

        System.out.println("Account created successfully.");
    }

    // Find account
    public BankAccount findAccount(int accountNumber) {

        for (int i = 0; i < count; i++) {

            if (accounts[i].getAccountNumber() ==
accountNumber) {
                return accounts[i];
            }
        }

        return null;
    }

    // Deposit money
    public void depositMoney(int accountNumber, double amount)
{

        BankAccount account = findAccount(accountNumber);

        if (account != null) {
            account.deposit(amount);
        } else {
```

```
            System.out.println("Account not found.");
        }
    }

    // Withdraw money
    public void withdrawMoney(int accountNumber, double amount)
{

        BankAccount account = findAccount(accountNumber);

        if (account != null) {
            account.withdraw(amount);
        } else {
            System.out.println("Account not found.");
        }
    }

    // Check balance
    public void checkBalance(int accountNumber) {

        BankAccount account = findAccount(accountNumber);

        if (account != null) {
            System.out.println("Current Balance: Rs. "
                    + account.getBalance());
        } else {
            System.out.println("Account not found.");
        }
    }

    // Display account details
    public void displayAccount(int accountNumber) {

        BankAccount account = findAccount(accountNumber);

        if (account != null) {
```

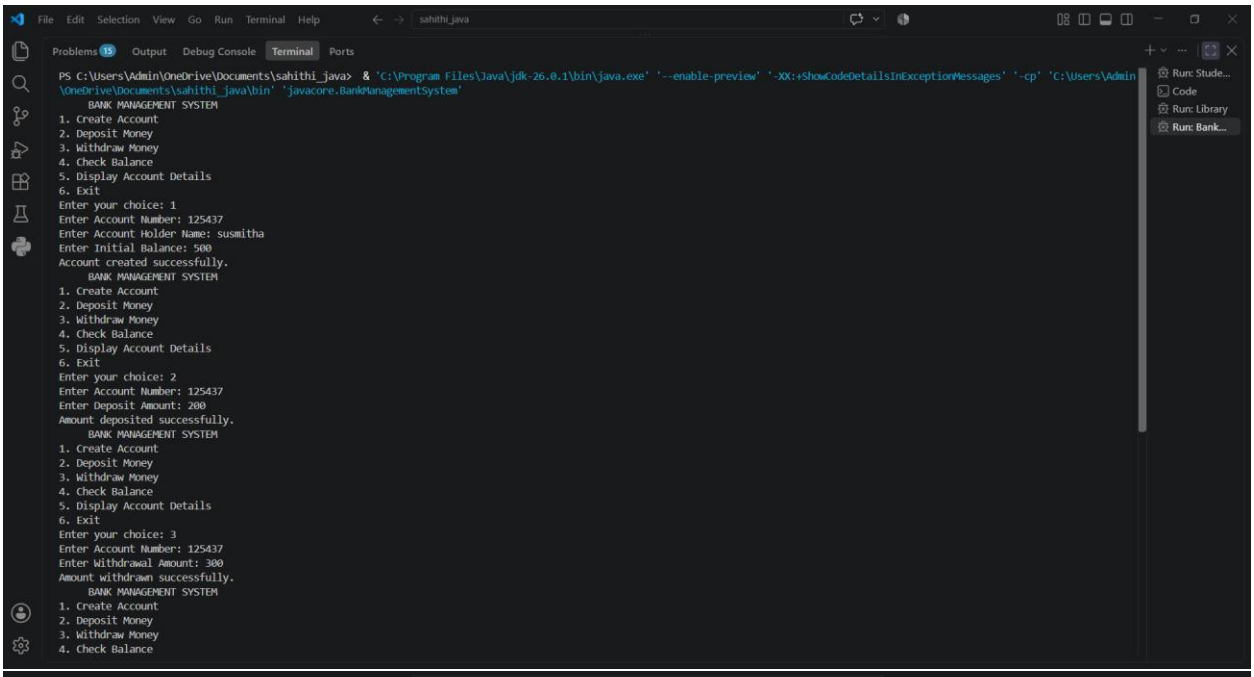
```
            account.displayDetails();
        } else {
            System.out.println("Account not found.");
        }
    }
}


// Main Class
public class BankManagementSystem {

    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Create Bank object
        // Maximum 100 accounts
        Bank bank = new Bank(100);

        int choice;

        do {
            System.out.println("     BANK MANAGEMENT SYSTEM");
            System.out.println("1. Create Account");
            System.out.println("2. Deposit Money");
            System.out.println("3. Withdraw Money");
            System.out.println("4. Check Balance");
            System.out.println("5. Display Account Details");
            System.out.println("6. Exit");
            System.out.print("Enter your choice: ");
            choice = sc.nextInt();

            switch (choice) {

                case 1:
```

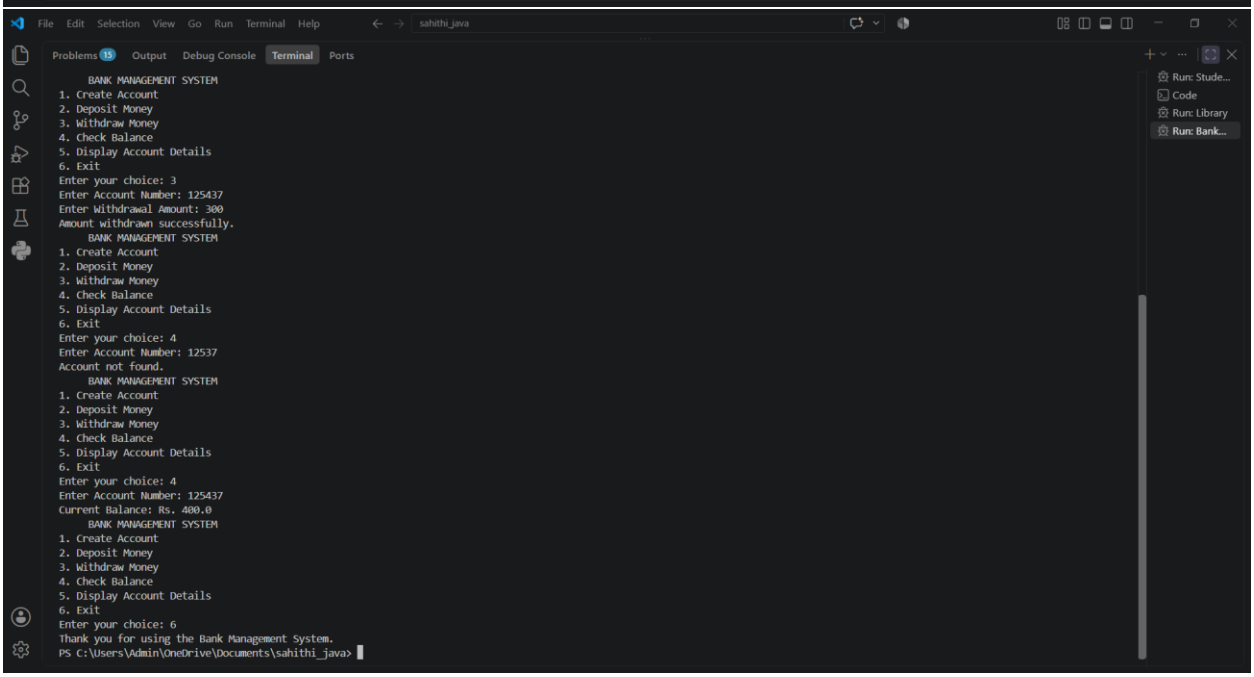
```
                    System.out.print("Enter Account Number: ");
                    int accountNumber = sc.nextInt();

                    sc.nextLine();

                    System.out.print("Enter Account Holder
Name: ");
                    String name = sc.nextLine();

                    System.out.print("Enter Initial Balance:
");
                    double balance = sc.nextDouble();

                    bank.createAccount(
                            accountNumber,
                            name,
                            balance
                    );

                    break;


                case 2:

                    System.out.print("Enter Account Number: ");
                    accountNumber = sc.nextInt();

                    System.out.print("Enter Deposit Amount: ");
                    double depositAmount = sc.nextDouble();

                    bank.depositMoney(
                            accountNumber,
                            depositAmount
                    );

                    break;
```

```
                case 3:

                    System.out.print("Enter Account Number: ");
                    accountNumber = sc.nextInt();

                    System.out.print("Enter Withdrawal Amount:
");
                    double withdrawAmount = sc.nextDouble();

                    bank.withdrawMoney(
                            accountNumber,
                            withdrawAmount
                    );

                    break;


                case 4:

                    System.out.print("Enter Account Number: ");
                    accountNumber = sc.nextInt();

                    bank.checkBalance(accountNumber);

                    break;


                case 5:

                    System.out.print("Enter Account Number: ");
                    accountNumber = sc.nextInt();

                    bank.displayAccount(accountNumber);
```

```
                    break;


                case 6:

                    System.out.println(
                        "Thank you for using the Bank
Management System."
                    );

                    break;


                default:

                    System.out.println(
                        "Invalid choice. Please try again."
                    );
            }

        } while (choice != 6);

        sc.close();
    }
}
```

Code screenshot:


File Edit Selection View Go Run Terminal Help
sahithi_java
Problems 15 Output Debug Console Terminal Ports
PS C:\Users\Admin\OneDrive\Documents\sahithi_java> & 'C:\Program Files\Java\jdk-26.0.1\bin\java.exe' '--enable-preview' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\Users\Admin\OneDrive\Documents\sahithi_java\bin' 'javacore.BankManagementSystem'
BANK MANAGEMENT SYSTEM
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Check Balance
5. Display Account Details
6. Exit
Enter your choice: 1
Enter Account Number: 125437
Enter Account Holder Name: susmitha
Enter Initial Balance: 500
Account created successfully.
BANK MANAGEMENT SYSTEM
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Check Balance
5. Display Account Details
6. Exit
Enter your choice: 2
Enter Account Number: 125437
Enter Deposit Amount: 200
Amount deposited successfully.
BANK MANAGEMENT SYSTEM
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Check Balance
5. Display Account Details
6. Exit
Enter your choice: 3
Enter Account Number: 125437
Enter Withdrawal Amount: 300
Amount withdrawn successfully.
BANK MANAGEMENT SYSTEM
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Check Balance
Run: Stude...
Code
Run: Library
Run: Bank...


File Edit Selection View Go Run Terminal Help
sahithi_java
Problems 15 Output Debug Console Terminal Ports
BANK MANAGEMENT SYSTEM
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Check Balance
5. Display Account Details
6. Exit
Enter your choice: 3
Enter Account Number: 125437
Enter Withdrawal Amount: 300
Amount withdrawn successfully.
BANK MANAGEMENT SYSTEM
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Check Balance
5. Display Account Details
6. Exit
Enter your choice: 4
Enter Account Number: 12537
Account not found.
BANK MANAGEMENT SYSTEM
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Check Balance
5. Display Account Details
6. Exit
Enter your choice: 4
Enter Account Number: 125437
Current Balance: Rs. 400.0
BANK MANAGEMENT SYSTEM
1. Create Account
2. Deposit Money
3. Withdraw Money
4. Check Balance
5. Display Account Details
6. Exit
Enter your choice: 6
Thank you for using the Bank Management System.
PS C:\Users\Admin\OneDrive\Documents\sahithi_java>
Run: Stude...
Code
Run: Library
Run: Bank...